

Chapter 0: Foundations and Prerequisites

Pre-LLM Multi-Agent Theory, Transformer Mechanics, and the Kernel Gap

The Port Daddy Authors

September 2026

Express lane: Before constructing an operating kernel for autonomous systems, we must establish the physical and theoretical ground upon which modern AI agents stand. This chapter establishes the five core foundations assumed by the subsequent eight chapters: (i) classical multi-agent systems and intentional agency (the BDI model, Hewitt–Agha actor concurrency, and FIPA speech acts); (ii) the physical mechanics of autoregressive transformers and the memory asymptotics of the key-value (KV) cache; (iii) the agent harness lifecycle, tool dispatch turn-loops, and hook-based observation injection; (iv) the compute topology separating frontier cloud reasoning from local host actuation; and (v) the architectural necessity of an external kernel, proving why generative models cannot act as their own operating system.

1 Introduction: The Lineage of Agency

Autonomous AI agents did not begin with large language models. The problem of coordinating multiple independent computational processes, each pursuing distinct local goals under incomplete information, has occupied distributed systems and artificial intelligence research for more than four decades.

When modern developers observe an LLM invoke a bash shell, parse the JSON output, and plan its next move, they are observing the convergence of two previously separate research lineages:

1. **Symbolic Multi-Agent Systems (1975–2010):** Formal architectures for intentional agency, asynchronous message passing, contract negotiation, and communicative action.
2. **Statistical Connectionism (2017–2026):** Deep autoregressive sequence prediction trained over planet-scale code and text corpora, capable of flexible semantic synthesis and zero-shot reasoning.

The tension between these lineages defines the subject of this textbook. Modern frontier models supply unprecedented semantic reasoning, but possess neither persistent memory, monotonic security boundaries, nor physical execution guarantees. To scale from an isolated interactive assistant to an autonomous engineering fleet, we must place statistical models inside a rigorous structural envelope.

2 Classical Multi-Agent Foundations

Three classical formulations provide the mathematical and conceptual language for reasoning about multi-agent systems: the BDI model of mental states, the Actor model of asynchronous concurrency, and the theory of communicative speech acts.

2.1 The BDI Architecture (Beliefs, Desires, Intentions)

Formalized by Rao and Georgeff [?, ?] following Michael Bratman’s philosophical account of practical human deliberation [?], the **BDI** architecture models an agent as a bounded rational system governed by three primary mental attitudes:

- **Beliefs ($\mathcal{B}$):** The agent’s information about the state of the world, itself, and other agents. In an LLM agent, $\mathcal{B}$ is represented in-flight by the contents of the context window (system prompts, user instructions, file listings, tool execution outputs).
- **Desires ($\mathcal{D}$):** The motivational states or potential objectives allocated to the agent. A software development agent may desire to “fix bug #412”, “maintain 100% test coverage”, and “minimize execution latency”.
- **Intentions ($\mathcal{I}$):** The subset of desires to which the agent has actively committed resources and execution time. Commitments persist across discrete actions until achieved, recognized as impossible, or superseded by higher-priority events.

Design Invariant (Commitment Persistence).

An agent’s intention $\iota \in \mathcal{I}$ must not be abandoned merely because a single step incurs a transient tool failure. Intention reconsideration is an explicit state transition triggered by invariant failure or objective discharge, not an accidental consequence of context window truncation.

In classical BDI interpreters (such as PRS, dMARS, or Jason), intention maintenance was regimented by a deterministic execution cycle. In an LLM agent, where deliberation occurs through autoregressive generation, intentions are fragile: attention dilution or misleading intermediate tool errors routinely cause models to wander into unrelated sub-tasks—a pathology known as *intention drift*.

2.2 The Hewitt–Agha Actor Model

Concurrency in multi-agent environments cannot safely rely on shared-memory synchronization with raw threading primitives. The theoretical foundation of modern agent message fabrics is the **Actor Model**, pioneered by Carl Hewitt [?] and formalized by Gul Agha [?].

In the Actor model, the primitive unit of computation is the *Actor*. Each actor possesses:

1. Private, unshared internal state.
2. A sequential mailbox queue.
3. The ability to send a finite number of messages to addresses it possesses.
4. The ability to create a finite number of new actors.
5. The ability to designate the replacement behavior for the next message.

$$\text{Actor}(S, m) \xrightarrow{\text{atomic}} (S', \{m_1, m_2, \dots, m_k\}, \{\alpha_{\text{new},1}, \dots, \alpha_{\text{new},j}\}) \quad (1)$$

The Actor model enforces *share-nothing isolation*. Modern agent harnesses mirror this structure: an autonomous agent runs in an isolated operating process or sandbox, interacts with peers exclusively via structured message buses (such as Swarm Radio or Relay), and delegates tasks by spawning subagents with scoped authority.

2.3 Speech Acts and Communicative Performatives

Agents do not merely transmit unstructured data; their messages constitute actions that alter social and legal commitments. Based on Austin’s speech act theory [?] and Searle’s philosophy of language [?], the Foundation for Intelligent Physical Agents (**FIPA**) standardized Agent Communication Language (**ACL**) performatives [?]:

- **inform**(i, j, ϕ): Agent i asserts to j that proposition ϕ holds.
- **request**(i, j, a): Agent i asks j to perform action a .
- **propose**(i, j, a, c): Agent i offers to perform action a under condition c .
- **agree**(i, j, a, c): Agent i commits to perform action a .
- **refuse**(i, j, a, r): Agent i declines request a with explanation r .

In this volume, communicative acts cease to be informal JSON payloads. They become cryptographically bound, verifiable transactions: an **agree** corresponds to an accepted Float Plan (ǵ??), while an **inform** must be backed by an append-only cryptographic receipt.

3 Transformer Mechanics and Memory Scaling

Modern agents deliberate by querying an autoregressive transformer. Understanding the computational and physical constraints of this inference engine is essential for building an operating kernel.

3.1 Autoregressive Inference and Attention Complexity

Given a sequence of input tokens $x_{1:t} = (x_1, \dots, x_t)$, a decoder-only transformer computes the conditional probability distribution over the vocabulary for the next token x_{t+1} :

$$P(x_{t+1} \mid x_1, \dots, x_t) = \text{softmax} \left(\frac{Q_t K_{1:t}^\top}{\sqrt{d_k}} \right) V_{1:t} \quad (2)$$

where Q, K, V are the query, key, and value projections. For a sequence of length T , computing full self-attention requires $O(T^2)$ floating-point operations across all query-key pairs.

3.2 The KV-Cache Asymptotics

During generation, past key and value vectors are cached in GPU memory to avoid recomputing past token representations at step $t+1$. For a model with L layers, N_h key-value heads, dimension per head d_h , and precision bytes b_p , the memory footprint of the KV-cache for a context of length T is:

$$\text{Memory}_{\text{KV}} = 2 \times L \times N_h \times d_h \times T \times b_p \quad \text{bytes} \quad (3)$$

For a typical 70-billion parameter model ($L = 80, N_h = 8, d_h = 128$, 16-bit FP16 precision $b_p = 2$), each token in the context consumes:

$$2 \times 80 \times 8 \times 128 \times 2 = 327,680 \text{ bytes/token} \approx 320 \text{ KB/token} \quad (4)$$

At a context length of $T = 128,000$ tokens, the KV-cache for a *single* agent process consumes approximately 40.96 GB of high-bandwidth VRAM—exceeding the entire parameter storage of smaller models.

⚠ Theorem (The Ephemeral Memory Limit).

The context window of a transformer is volatile, expensive, and subject to attention dilution (effective information retrieval degrades with sequence depth [?]). Consequently, long-term system state cannot be maintained by continuously appending tool execution logs to the prompt. Durable state must reside in an external, indexed, and ACID-compliant storage engine.

4 The Agent Harness Lifecycle

An LLM is a stateless function: it maps a sequence of integers to a probability distribution. An *Agent* is formed only when this stateless sampler is wrapped in an execution loop known as the **Agent Harness**.

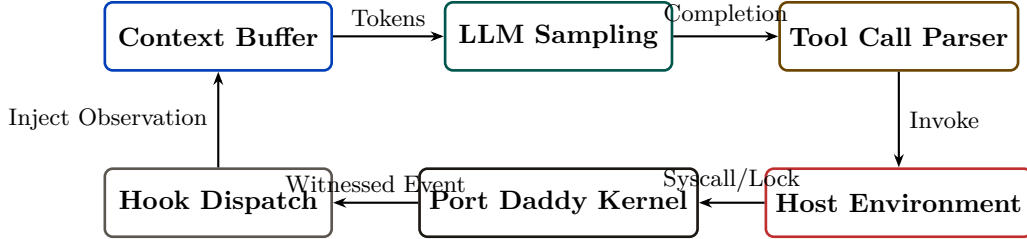


Figure 1: The closed agentic loop: LLM deliberation interleaved with kernel-mediated host actuation and observation feedback.

The harness operates as a deterministic finite-state machine driving an indeterminate generator:

1. **Pre-Invocation:** The harness assembles the system prompt, environment variables, tool schemas, and recent message history.
2. **Inference:** The prompt is transmitted to the inference provider (local or remote). Tokens are streamed until an end-of-turn or tool-use stop token is encountered.
3. **Interception & Validation:** The harness extracts the tool call arguments (e.g., `bash`, `write_file`, `http_request`). Before execution, the harness verifies permission parameters against local policies.
4. **Execution:** The command executes inside an operating sandbox or local process.
5. **Observation Injection:** The raw stdout/stderr output is sanitized, truncated if necessary to respect budget bounds, formatted as a tool-result block, and appended to the context.
6. **Evaluation:** The harness inspects whether terminal conditions (objective satisfied, unrecoverable crash, or budget exhaustion) are reached; if not, the cycle repeats.

5 Compute Topology: Frontier Cloud vs. Edge Actuation

Autonomous agent fleets are split across a radical computational asymmetry:

Attribute	Frontier Cloud Reasoning	Local Kernel / Host Edge
Workload	Multi-step decomposition, code synthesis	Deterministic ordering, file I/O, compilation
Latency	500 ms – 10 s per deliberation step	< 1 ms (POSIX locks, SQLite WAL writes)
Cost	\$1.00 – \$15.00 per million tokens	Local CPU cycles (effectively zero marginal cost)
Trust Boundary	Third-party cloud datacenter	Operator-controlled workstation / secure enclave
Failure Mode	Hallucination, semantic drift, timeout	Crash-stop, lock contention, disk exhaustion

Table 1: Asymmetry between cloud inference engines and local host execution surfaces.

Because cloud reasoning is high-latency and probabilistic, it is incapable of resolving local race conditions. If two cloud agents concurrently attempt to edit line 42 of the same file, relying on their internal world models to avoid collisions guarantees data corruption. The resolution must be executed locally by a synchronous, zero-latency authority.

6 The Kernel Gap: Why LLMs Cannot Be Their Own OS

A seductive but fatal fallacy in modern AI engineering is the belief that an LLM can serve as its own operating system kernel—that prompts such as “*Please do not modify files currently being edited by other agents*” are sufficient to preserve system integrity.

We term this discrepancy the **Kernel Gap**. Language models fail as operating systems across four fundamental dimensions:

6.1 1. Non-Determinism vs. Invariant Regimentation

An operating system kernel must enforce *regimented invariants*: physical guarantees that cannot be bypassed by any user process (e.g., memory page protection via MMU hardware). Prompts are mere suggestions. Even fine-tuned models exhibit a non-zero probability of instruction failure, hallucination, or adversarial prompt injection. A security policy enforced solely by a prompt is an advisory boundary, not an invariant.

6.2 2. Concurrency and the Mutual Exclusion Anomaly

In any concurrent system with two or more uncoordinated writers, atomic mutual exclusion is required to prevent race conditions. Without an external serialized lock broker, concurrent LLM agents exhibit the classic lost-update anomaly: Agent A reads version v_0 , Agent B reads v_0 , Agent A writes v_A , and Agent B subsequently overwrites with v_B , silently destroying Agent A’s work without compiler or runtime warning.

6.3 3. Attenuation and Privilege Escalation

In classical capability-based operating systems (e.g., KeyKOS, EROS, Capsicum), authority flows monotonically downward: a child process can never possess more capability than its parent. When an LLM delegates sub-tasks by synthesizing new prompt instructions, authority frequently inflates due to broad, imprecise system instructions. True privilege containment requires cryptographically unforgeable, attenuated tokens (Harbor Cards, Chapter 2).

6.4 4. Mortality and Economic Accountability

Subprocesses are transient. An agent can crash, get SIGKILLed, or reset its context, discarding all local history. If reputation, debt, and liability exist only in process RAM, defecting agents face zero consequence for bad actions. Accountability requires persistent outcome ledgers, Merkle-backed event histories, and economic escrows that survive the death of the acting process (Chapters 5–7).

7 Chapter Map and Structure of the Book

The remainder of this textbook is organized into eight cumulative chapters across four distinct operational levels:

- **Chapter 1: The Single-Writer Kernel (Ground Truth).** We construct the minimal deterministic local kernel using SQLite WAL mode, establishing serializability, synchronous state decidability, and crash durability across process failures.

- **Chapter 2: The Anchor Protocol (Capability Delegation).** We design cryptographically verifiable capability tokens (Harbor Cards) and prove that delegated authority can only narrow at every hop.
- **Chapter 3: The Sealed Harbor (Confinement & Leakage).** We construct isolated execution harbors that bound information leakage between distrustful principals under strict egress quotas.
- **Chapter 4: The Legible Swarm (Human Supervision).** We derive the information-theoretic cost of supervisory attention, proving how digests must retain evidence paths without drowning the operator in raw telemetry.
- **Chapter 5: From Spawn to Person (Durable Agency).** We define persistent agent identity, append-only outcome ledgers, and reputation mechanisms that survive process restarts.
- **Chapter 6: The Harbor Economy (Markets & Clearing).** We establish bilateral clearing, underwriting reserves, and dynamic pricing for scarce host resources.
- **Chapter 7: The Bonded Commons (Game-Theoretic Governance).** We formalize claim signaling as an incentive-compatible repeated game, proving when cooperation is sustained under bonded escrow.
- **Chapter 8: The Federated Harbor (Cross-Machine Consensus).** We extend governance across distrustful host boundaries via witness logs and sheaf-theoretic consistency without central sovereignty.

References

- [1] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, Cambridge, MA, 1986.
- [2] J. L. Austin. *How to Do Things with Words*. Oxford University Press, 1962.
- [3] M. E. Bratman. *Intention, Plans, and Practical Reason*. Harvard University Press, Cambridge, MA, 1987.
- [4] Foundation for Intelligent Physical Agents. FIPA ACL Communicative Act Library Specification. Technical Report SC00037J, FIPA, 2002.
- [5] C. Hewitt, P. Bishop, and R. Steiger. A universal modular ACTOR formalism for artificial intelligence. In *Proc. 3rd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 235–245, 1973.
- [6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024.
- [7] A. S. Rao and M. P. Georgeff. Modeling rational agents within a BDI-architecture. In *Proc. 2nd International Conference on Principles of Knowledge Representation and Reasoning (KR)*, pages 473–484, 1991.
- [8] A. S. Rao and M. P. Georgeff. BDI agents: From theory to practice. In *Proc. 1st International Conference on Multi-Agent Systems (ICMAS)*, pages 312–319, 1995.
- [9] J. R. Searle. *Speech Acts: An Essay in the Philosophy of Language*. Cambridge University Press, 1969.